COMENIUS UNIVERSITY IN BRATISLAVA
FACULTY OF MATHEMATICS PHYSICS AND INFORMATICS

A MODULAR CLUSTER FOR AI RESEARCH IN RESOURCE-CONSTRAINED ACADEMICENVIRONMENTS

Master thesis

2026 Bc. Filip Sršeň

COMENIUS UNIVERSITY IN BRATISLAVA
FACULTY OF MATHEMATICS PHYSICS AND INFORMATICS

A MODULAR CLUSTER FOR AI RESEARCH IN RESOURCE-CONSTRAINED ACADEMICENVIRONMENTS

Master thesis

Study program:	Applied informatics
Branch of study:	Applied informatics
Department:	Department of Applied Informatics
Supervisor:	Mgr. Ján Kľuka, PhD.

Bratislava, 2026

Bc. Filip Sršeň

47430683

Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

ZADANIE ZÁVEREČNEJ PRÁCE

Meno a priezvisko študenta: Bc. Filip Sršeň
Študijný program: aplikovaná informatika (Jednoodborové štúdium, magisterský II. st., denná forma)
Študijný odbor: informatika
Typ záverečnej práce: diplomová
Jazyk záverečnej práce: anglický
Sekundárny jazyk: slovenský

Názov: A Modular Cluster for AI Research in Resource-Constrained Academic Environments
Modulárny klaster pre výskum v AI v akademickom prostredí s obmedzenými zdrojmi

Anotácia: Významná časť výskumu na našej katedre sa opiera o metódy strojového učenia, no dostupná výpočtová infraštruktúra je obmedzená a náročná na správu. Výskumníci pravidelne narážajú na problémy so zdieľaním hardvérových zdrojov, efektívnou prácou s dátovými súbormi a presunom experimentov medzi rôznymi prostrediami. Súčasné postupy – ako manuálne plánovanie, udržiavanie samostatných softvérových inštalácií či kopírovanie dát medzi servermi – komplikujú prácu a znižujú jej produktivitu.

Táto práca sa zameriava na riešenie týchto výziev skúmaním možností návrhu modulárneho klastra prispôsobeného potrebám katedry využitím súčasných technológií na virtualizáciu, kontajnerizáciu a správu klastrov implementovaných ako slobodný softvér/softvér s otvoreným zdrojovým kódom (Proxmox, Docker, Kubernetes, SLURM a pod.). Zámerom je navrhnúť riešenie zlepšujúce prístupnosť, škálovateľnosť a efektivitu práce s ohľadom praktické obmedzenia dostupných zdrojov a pracovné postupy používateľov. Implementácia má demonštrovať, ako tieto môžu uvedené technológie podporovať akademický výskum poskytovaním základu, ktorý uľahčuje spoluprácu, zjednodušuje pracovné postupy a zvyšuje reprodukovateľnosť experimentov strojového učenia.

Cieľ: Hlavným cieľom tejto práce je navrhnúť a implementovať modulárny výpočtový klaster prispôsobený technickým a prevádzkovým obmedzeniam akademického výskumného prostredia. Okrem zabezpečenia zdieľanej výpočtovej kapacity bude klaster slúžiť ako platforma integrujúca do jedného celku plánovanie výpočtových úloh, správu prostredí a sledovanie experimentov. Implementácia systému ukáže, ako možno moderné orchestračné technológie prispôsobiť menšej akademickej infraštruktúre s cieľom zlepšiť reprodukovateľnosť a spoľahlivosť, zjednodušiť údržbu a efektívne využívať dostupné výpočtové zdroje.

Literatúra:
Yoo, A.B., Jette, M.A., Grondona, M (2003). SLURM: Simple Linux Utility for Resource Management. In: JSSPP 2003. Springer.
Domingus, J., Arundel, J. (2022): Cloud Native DevOps with Kubernetes, 2nd Edition. O'Reilly.

47430683

Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

Salatino, M. (2024): Platform Engineering on Kubernetes. Manning.
Spišaková, V., Klusáček, D., Hejtmánek, L. (2023). Using Kubernetes in Academic Environment: Problems and Approaches. In: JSSPP 2022. LNCS 13592. Springer
Tim Wickberg (2025). Slinky: Slurm In Kubernetes, Performant AI And HPC Workload Management. In: KubeCon Europe.
Petrosyan, D.A. (2025). Scheduling ML and HPC Jobs with Shoc Platform over Kubernetes. Phys. Part. Nuclei 56, 1581–1585.
Chazapis, A., Nikolaidis, F., Marazakis, M., Bilas, A. (2023). Running Kubernetes Workloads on HPC. In: ISC High Performance 2023. LNCS 13999. Springer.
Misale, C. et al. (2022). Towards Standard Kubernetes Scheduling Interfaces for Converged Computing. In: SMC 2021. CCIS 1512. Springer.

Vedúci: Mgr. Ján Kľuka, PhD.
Katedra: FMFI.KAI - Katedra aplikovanej informatiky
Vedúci katedry: doc. RNDr. Tatiana Jajcayová, PhD.

Dátum zadania: 31.10.2025

Dátum schválenia: 11.11.2025

prof. RNDr. Roman Ďurikovič, PhD.
garant študijného programu

.....
študent

.....
vedúci práce

I hereby declare that I have written this thesis by myself, only with help of referenced literature, under the careful supervision of my thesis advisor.

. . . . . .

Bratislava, 2026 Bc. Filip Sršeň

Acknowledgement

TODO

Abstract

TODO

Keywords: space debris, machine learning, space object classification

Abstrakt

TODO

Kľúčové slová: vesmírny odpad, strojové učenie, klasifikácia vesmírnych objektov

Contents

1 Introduction **1**

1.1 Motivation 1

1.2 Problem statement 1

1.3 Thesis Contributions 1

1.4 Thesis Structure 1

2 Background & Related Work **2**

2.1 Cloud-Native Orchestration 2

2.2 Batch vs. Interactive Computing 2

2.3 Distributed Machine Learning 2

2.4 State of the Art 2

3 System Architecture Design **3**

3.1 Introduction 3

3.1.1 Design Objectives 3

3.1.2 Design Philosophy 3

3.2 High-Level System Model 3

3.2.1 Layered Abstraction Framework 3

3.2.2 System Interaction Model 3

3.2.3 The Modular Interconnectivity 3

3.3 Detailed Component Architecture 3

3.3.1 Identity and Governance Layer 3

3.3.2 Research Interface and Environment Provisioning 4

3.3.3 Distributed Orchestration and Scheduling Engine 4

3.3.4 Experiment Lifecycle and Metadata Management 4

3.4 Design for Resource Constraint Mitigation 4

3.4.1 Advanced Scheduling Strategies 4

3.4.2 Elasticity and Resource Elasticity 4

3.5 Summary 4

4 Implementation & Technical Realization **5**
4.1 Deployment Strategy 5
4.2 Implementing the Orchestration Engine 5
4.2.1 Configuring Volcano for Academic Constraints 5
4.2.2 Integrating Kube-Ray for Distributed Scaling 5
4.3 Building the Research Interface 5
4.4 Data and Lifecycle Automation 5

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Motivation

The widening gap between industry-scale AI compute and the heterogeneous, resource-constrained environments found in academia.

1.2 Problem statement

The complexity of managing distributed AI workloads and the difficulty of providing a "single-pane-of-glass" interface for researchers without heavy DevOps overhead.

1.3 Thesis Contributions

- A modular, Kubernetes-native platform for AI research.
- An intelligent scheduling layer for resource-efficient batch processing.
- A distributed executor for scaling research workloads.

1.4 Thesis Structure

Overview of the subsequent chapters.

Chapter 2

Background & Related Work

2.1 Cloud-Native Orchestration

Fundamentals of Kubernetes and container-based workloads.

2.2 Batch vs. Interactive Computing

The challenge of scheduling high-throughput AI jobs alongside interactive researcher notebooks.

2.3 Distributed Machine Learning

Paradigms of data and model parallelism and the need for specialized orchestrators.

2.4 State of the Art

Review of existing frameworks (e.g., Kubeflow) and identifying the "gap" (e.g., complexity, lack of modularity for specialized academic clusters).

Chapter 3

System Architecture Design

3.1 Introduction

3.1.1 Design Objectives

Modularity, Scalability, and Resource Efficiency.

3.1.2 Design Philosophy

Cloud-native, microservice-based approach using Kubernetes.

3.2 High-Level System Model

3.2.1 Layered Abstraction Framework

Identity, Interface, Orchestration, and Lifecycle.

3.2.2 System Interaction Model

End-to-end workflow from auth to distributed training.

3.2.3 The Modular Interconnectivity

Decoupling via standardized interfaces (APIs/K8s CRDs).

3.3 Detailed Component Architecture

3.3.1 Identity and Governance Layer

Keycloak integration for multi-tenancy.

3.3.2 Research Interface and Environment Provisioning

JupyterHub and Harbor (private registry).

3.3.3 Distributed Orchestration and Scheduling Engine

Volcano

Batch scheduling and gang-scheduling for AI workloads.

Kube-Ray

Distributed compute scaling.

3.3.4 Experiment Lifecycle and Metadata Management

MLflow integration for tracking metrics/artifacts.

3.4 Design for Resource Constraint Mitigation

3.4.1 Advanced Scheduling Strategies

Volcano queueing and priority mechanisms.

3.4.2 Elasticity and Resource Elasticity

K8s/Ray dynamic scaling.

3.5 Summary

Concluding synthesis of modular design meets the problem statement.

Chapter 4

Implementation & Technical Realization

4.1 Deployment Strategy

Using GitOps and Helm-based manifests to manage the cluster lifecycle.

4.2 Implementing the Orchestration Engine

4.2.1 Configuring Volcano for Academic Constraints

Priority queues and gang-scheduling.

4.2.2 Integrating Kube-Ray for Distributed Scaling

Distributed compute scaling.

4.3 Building the Research Interface

How the API Gateway and JupyterHub provide the unified interface for the end-user.

4.4 Data and Lifecycle Automation

The integration of Harbor for image stability and MLflow for scientific experiment tracking.